Tokenization

Advanced NLP M26

Manish Shrivastava + Aparajitha

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

August 10, 13, 2026

Transformer

Where are we?

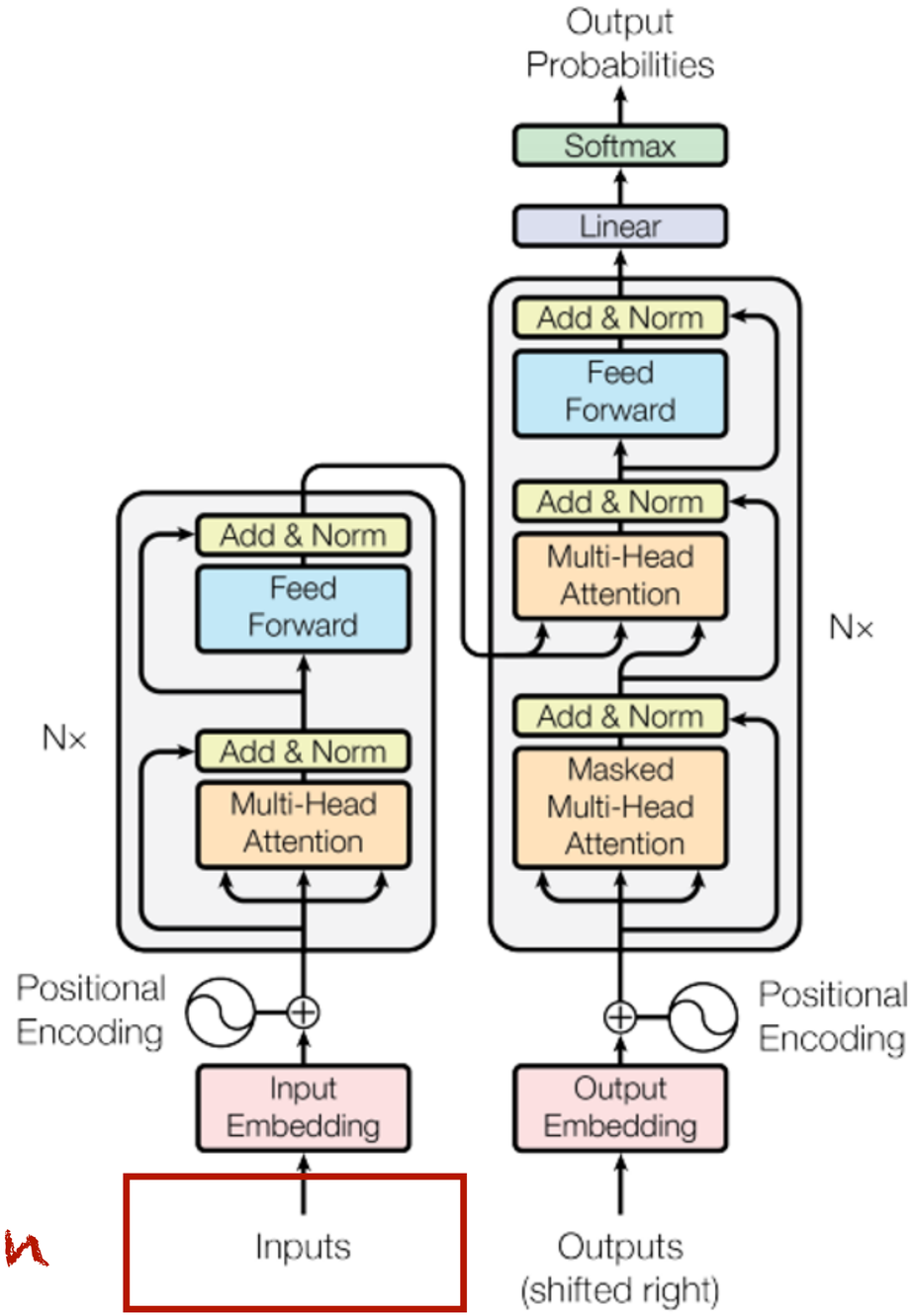


Figure 1: The Transformer - model architecture.

Tokenization

Tokens

- Atomic units of text
- Are words tokens? (~Whitespace Tokenization)

	1	2	3	4	5	6	7	8	9	10	11	12	13
1	In	the	winter	that	seat	is	close	enough	to	the	radiator		
2	to	remain	**warm**	,	and	yet	not	so	close	as	to	cause	
3	**perspiration**	.	In	the	summer	***it's***	directly	in	the	path	of	a	
4	cross	breeze	created	by	open	windows	**there**	,	and	**there**	.	It	faces
5	the	television	at	an	angle	that	is	neither	**direct**	,	thus	discouraging	
6	**conversation**	,	nor	so	far	wide	to	create	a	parallax	**distortion**	,	I
7	could	go	**on**	,	but	I	think	***I've***	made	my	**point**	.	

Tokens - Challenges

What is a token?

- Hypenation
- Numbers
- Dates
- Abbreviations
- Emojis
- Code Switching/Mixing
- Speech Transcripts
  - Disfluencies/Fillers

Tokens - Challenges

- Orthographies without word boundaries
  - Logographic scripts - Chinese, Japanese
  - Abugidas - Thai, Lao, Khmer, Burmese

- Are tokens finite?
  - Out-of-vocabulary

Morphemes

- Root word / single unit of meaning
- Chinese - words are composed of characters (Hanzi)
  - each character is a morpheme andcan be treated as a word
- What about Japanese and Thai?
- What about agglutinative languages like Telugu?

Tokenization

Why do we tokenize?

- Efficiency
  - Reduced Vocabulary
  - Simpler and efficient computation

Tokenization

Levels

- Character
- Word
- Sub-word
- Sentence

Character Level

Encoding

- ASCII
- Unicode
  - UTF - 8 (Unicode Transformation Format 8)

- Requires 1-3 bytes per character

Tokenization

Methods - Word/Sub-Word

- Whitespace
- Rule-based
- Statistical
- No tokenization

Tokenization

Challenges

- Whitespace - Scripts without spaces
- Rule-based - Niche uses; language dependent

Statistical

Data-based methods

- WordPiece
- ULM -> SentencePiece
- BPE

WordPiece

Algorithm

1. Initialize vocab with individual characters
2. Calculate likelihood of all adjacent symbol pairs in the corpus
3. Merge the pair, which maximizes likelihood, into a single token
4. Update the corpus
5. Repeat till desired vocabulary size

WordPiece uses greedy longest-match strategy

Popularized for subwords in NLP by BERT

Schuster, M., & Nakajima, K. (2012, March). Japanese and korean voice search. In 2012 IEEE international conference on acoustics, speech and signal processing (ICASSP) (pp. 5149-5152). IEEE.

WordPiece

Limitations

- Unknown characters map to <UNK>
- Requires training data to cover all characters
- Challenges with Morphologically rich, agglutinative, and no-word boundary languages
- Performance degradation on domain-specific data (Medical/ Legal)

Unigram Language Model (ULM)

Algorithm

- Reasonably big seed vocabulary
- Repeat until desired vocabulary size
  - Optimize subword occurrence probability with EM
  - Compute the loss for each subword (How likely the likelihood is reduced if subword removed from vocab)
  - Sort by loss and keep top n% of subwords
- Note: subwords consisting of a single character are not removed to avoid out-of-vocabulary

Kudo, T. (2018, July). Subword regularization: Improving neural network translation models with multiple subword candidates. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 66-75).

SentencePiece

- Raw input stream includes space as a character
- ULM to construct vocabulary

*Implementation of ULM

Kudo, T., & Richardson, J. (2018, November). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Proceedings of the 2018 conference on empirical methods in natural language processing: System demonstrations (pp. 66-71).

ULM/SentencePiece

Challenges

- Computational cost of training
- Defining good seed vocabulary
- Pruning % is a hyperparameter
- Not-deterministic

Byte Pair Encoding (BPE)

History

- Simple data compression technique
- Iteratively replace the most frequent pair of bytes in a sequence with a single, unused byte

Gage, P. (1994). A new algorithm for data compression. The C Users Journal, 12(2), 23-38.

Byte Pair Encoding (BPE)

Algorithm for Word Segmentation

- Initialize vocabulary with characters
- Iteratively count symbol pairs
- Replace most frequent pair with merged symbol
  - eg. if ‘t’ ‘h’ occur most together they are replaced by ‘th’
- Each merge produces a new character n-gram
- Frequent character n-grams (or whole words) are eventually merged into a single symbol

Sennrich, R., Haddow, B., & Birch, A. (2016, August). Neural machine translation of rare words with subword units. In Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers) (pp. 1715-1725).

Byte Pair Encoding (BPE)

Advantages

- Simple and fast
  - No EM or probabilistic modeling required
- Handles OOV
- Deterministic Segmentation
- Language and domain agnostic

Byte Pair Encoding (BPE)

Challenges

- Greedy step-wise merge can be globally suboptimal
- Sensitive to corpus
- Doesn’t align with linguistic units
- Inefficient for rare and complex words

Byte level BPE

- Byte stream instead of characters
  - UTF-8 Encoding instead of Unicode characters

Sennrich, R., Haddow, B., & Birch, A. (2016, August). Neural machine translation of rare words with subword units. In Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers) (pp. 1715-1725).

Byte level BPE

Advantages

- No OOV
- Language and script agnostic
- Robustness to noisy data/unusual symbols
- Simplifies preprocessing
  - No language-specific normalization or Unicode handling

Byte level BPE

Challenges

- BPE disadvantages - Greedy, corpus sensitive
- Non-Latin scripts (Chinese, Japanese) end up with disproportionally longer sequences than Latin-scripts
  - Concerns in multilingual cost
- Merges harder to interpret

Parity-aware BPE

- Fix disproportionally longer sequences for non-Latin scripts
- Replaces this global objective with a max–min criterion
  - Selects merge that most improves the worst-compressed language

* Not used in any known deployed models; ACL 2026

Foroutan, N., Meister, C., Paul, D., Niklaus, J., Ahmadi, S., Bosselut, A., & Sennrich, R. (2026, July). Parity-aware byte-pair encoding: Improving cross-lingual fairness in tokenization. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 7514-7538).

No Tokenization

Motivation

- Current tokenization approaches are neither human-like nor grounded in linguistic concepts
  - Humans think in concepts
- Number and Character Understanding
- Tokenization sensitive to modality, domain, and input noise

No Tokenization

Approaches

- **Byte or character models** that operate on raw bytes or characters and learn sequence patterns directly
- **Continuous input encoders** that convert signals into learned embeddings without tokens
- **Hybrid models** that ingest raw input but internally learn chunking or segmentation as part of training

No Tokenization

Research

- MegaByte

Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., & Lewis, M. (2023). Megabyte: Predicting million-byte sequences with multiscale transformers. *Advances in Neural Information Processing Systems*, *36*, 78808-78823.

- MambaByte

Wang, J., Gangavarapu, T., Yan, J. N., & Rush, A. M. (2024). Mambabyte: Token-free selective state space model. *First Conference on Language Modelling*

- SpaceByte

Slagle, K. (2024). Spacebyte: Towards deleting tokenization from large language modeling. *Advances in Neural Information Processing Systems*, *37*, 124925-124950.

- Byte Latent Transformer

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 9238-9258).

Byte Latent Transformer

- Uses entropy-based grouping to dynamically segment bytes into patches
- Patches are created based on the entropy of the next byte
- 3 modules - Local Encoder, Latent Transformer, and Local Decoder
- Incorporates *byte n-gram embeddings* and a *cross-attention mechanism* to maximize information flow between the Latent Transformer and the byte-level modules

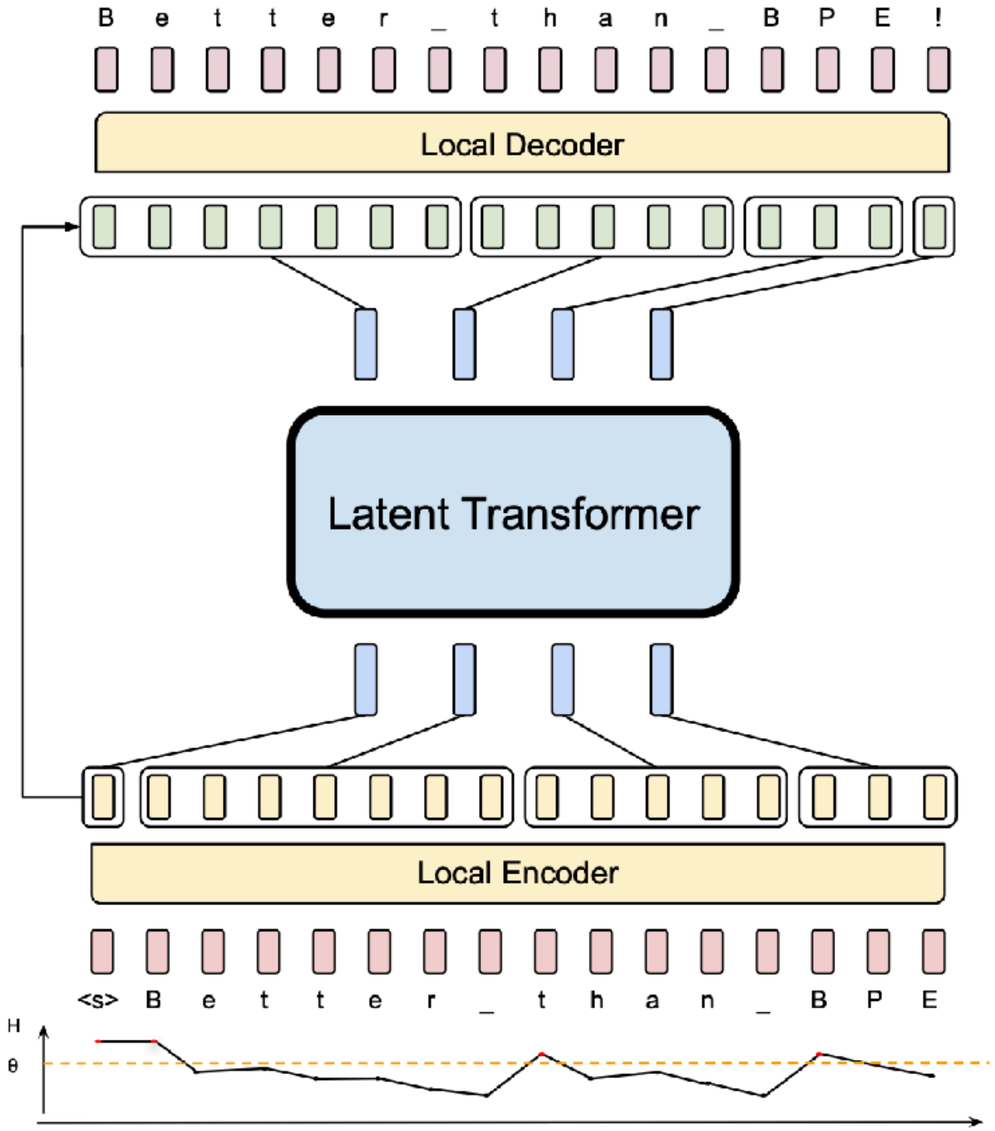
Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 9238-9258).

Byte Latent Transformer

- Lightweight Local Encoder that encodes input bytes into patch representations
- Computationally expensive Latent Transformer over patch representations
- Lightweight Local Decoder to decode the next patch of bytes
- Dynamically groups bytes into patches preserving access to the byte-level information

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 9238-9258).

Byte Latent Transformer



5. Small Byte-Level Transformer Makes **Next-Byte Prediction**

4. **Unpatching** to Byte Sequence via Cross-Attn

3. Large Latent Transformer **Predicts Next Patch**

2. Entropy-Based Grouping of Bytes Into **Patches** via Cross-Attn

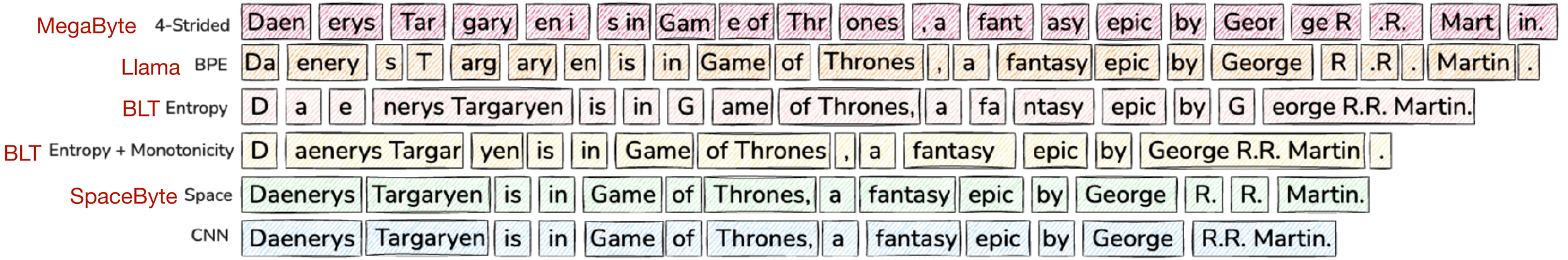
1. Byte-Level Small Transformer Encodes **Byte Stream**

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 9238-9258).

Byte Latent Transformer

Patching Schemes



INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY HYDERABAD

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 9238-9258).

Byte Latent Transformer

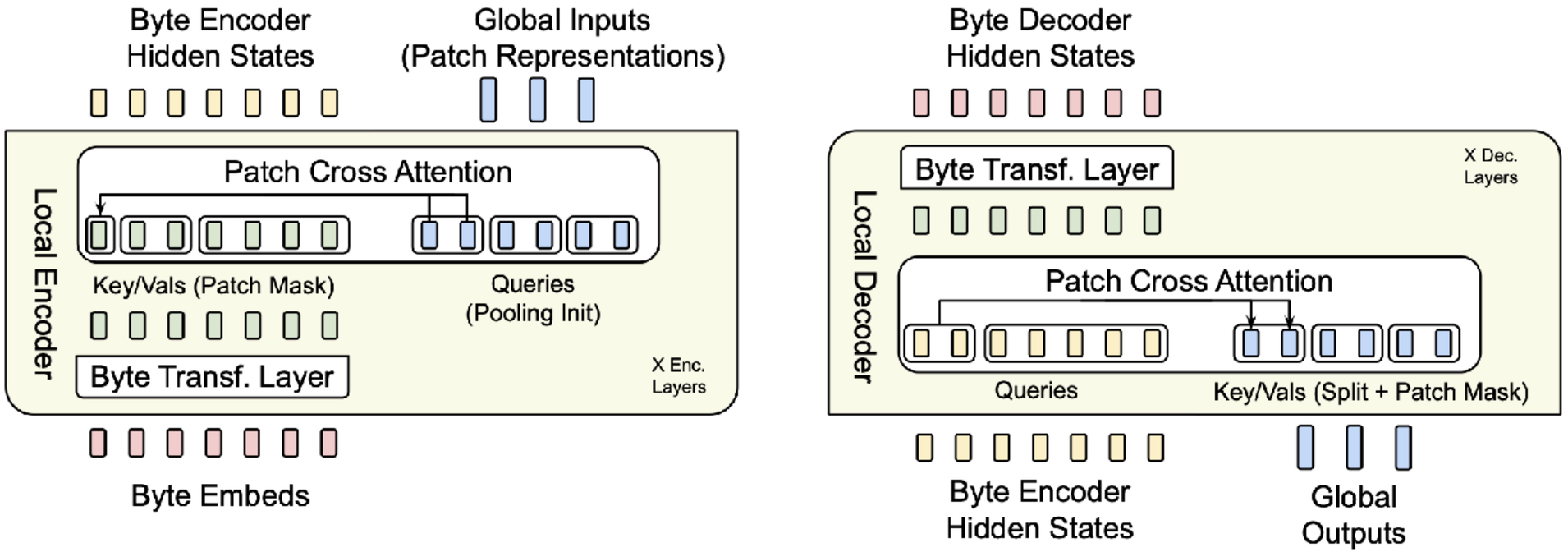


Figure 5 The local encoder uses a cross-attention block with patch representations as queries, and byte representations as keys/values to encode byte representations into patch representations. The local decoder uses a similar block but with the roles reversed i.e. byte representations are now the queries and patch representations are the keys/values. Here we use Cross-Attn $k = 2$.

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 9238-9258).

Byte Latent Transformer

Advantages

- Dynamically groups bytes into patches preserving access to the byte-level information unlike fixed vocabulary tokenization

Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., ... & Iyer, S. (2025, July). Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 9238-9258).